

POKHARA UNIVERSITY
GANDAKI COLLEGE OF ENGINEERING AND SCIENCE

LABORATORY REPORT
AGILE SOFTWARE DEVELOPMENT LAB

Lab No. 2
BESE III YEAR - VI SEM

Submitted By:

Pranish Poudel
Roll No: 26
Program: BESE ,2023

Submitted To:

Er. Rajendra Bahadur Thapa
Lecturer

Date: July 3, 2026

NAME OF EXPERIMENT

Exploring the key features of an Integrated Development Environment (IDE) using Visual Studio Code.

AIM

To examine and use the principal features of Visual Studio Code, namely Build configurations (Debug and Release), Breakpoints, Refactoring, Git-based source code collaboration, the Command Palette, and IntelliSense.

THEORY

An Integrated Development Environment (IDE) bundles every tool a programmer needs to write, run, debug, and organise software into one coherent application. Rather than juggling a separate editor, compiler, and debugger, the developer works inside a single interface, which raises productivity, lowers the cost of setup, and makes large codebases easier to navigate.

Visual Studio Code (VS Code) is a free, open-source, and lightweight IDE from Microsoft. Its popularity comes from cross-platform availability, a huge extension marketplace, and out-of-the-box support for modern workflows. Through extensions it handles many languages including Python, JavaScript, C++, and Java. The features studied in this lab are described below.

1. Build (Debug and Release): Building translates readable source code into a runnable program. A Debug build keeps extra debugging symbols and skips aggressive optimisation so that the running program can be traced back to the source, which makes fault-finding easier even though execution is slower. A Release build strips debugging information and applies strong optimisations to maximise speed and shrink the memory footprint for end users.
2. Breakpoints: A breakpoint is a marker set on a line of code that tells the debugger to pause execution when it reaches that line. While paused, the developer can read variable values, examine the call stack, and evaluate expressions. This is far more powerful than scattering print statements for diagnosing subtle runtime faults.
3. Refactoring: Refactoring means reshaping existing code to improve its internal structure and clarity without changing what it does externally. Typical actions include renaming symbols, extracting a block into its own function, and simplifying conditions. Done regularly, it keeps code clean and easy for the whole team to maintain.
4. Source Code Collaboration (Git): VS Code ships with built-in Git support. Directly from the editor a developer can initialise a repository, stage and commit changes, and synchronise with remotes such as GitHub, all without leaving for a separate terminal.
5. Command Palette: Opened with Ctrl+Shift+P, the Command Palette offers keyboard-driven access to almost every editor command. Instead of hunting through menus, the developer simply types the command name and runs it instantly.
6. IntelliSense: IntelliSense is the intelligent code-completion engine of VS Code. By analysing context, imported libraries, and types, it suggests variables, functions, and parameters as you type and shows signatures and documentation. This cuts down typos and syntax mistakes and speeds up coding.

OBJECTIVES

- To understand what an Integrated Development Environment is and why it is useful.
- To distinguish between Debug and Release build configurations.

- To learn how breakpoints assist during debugging.
- To grasp the idea and benefits of code refactoring.
- To use the built-in Git integration for version control.
- To operate the Command Palette for quick command access.
- To see how IntelliSense accelerates code completion.

ALGORITHM

1. Open Visual Studio Code and load a new project folder.
2. Create a source file such as app.js and write a short program with a few functions and variables.
3. Trigger IntelliSense by typing a partial keyword (for example "con" for console) and observe the completions and parameter hints.
4. Press Ctrl+Shift+P to open the Command Palette and run a command such as "Format Document".
5. Add a breakpoint by clicking in the gutter beside an executable line.
6. Launch the program in Debug mode from the Run and Debug panel and let it pause at the breakpoint.
7. While paused, inspect variable values and use Step Over to advance line by line.
8. Refactor by right-clicking a variable, choosing Rename Symbol, and confirming every occurrence updates.
9. Open the Source Control panel, initialise a repository, and stage the file.
- 10 Write a commit message, commit, then connect to a GitHub remote and push.
- 11 Note conceptually how a Release build would be optimised for performance.
- 12 Record the behaviour and usefulness of each feature explored.

CODE

JavaScript sample (for IntelliSense and breakpoints)

```

1 function add(a, b) {
2     return a + b;
3 }
4
5 function multiply(a, b) {
6     return a * b;
7 }
8
9 let x = 10;
10 let y = 20;
11 let sum = add(x, y);
12 let product = multiply(x, y);
13
14 console.log("Sum:", sum);
15 console.log("Product:", product);

```

Git commands used for collaboration

```

1 git init
2 git add .
3 git commit -m "Lab 2: exploring IDE features"
4 git remote add origin https://github.com/username/repository.git
5 git branch -M main

```

```
6 git push -u origin main
```

OUTPUT

Each feature of Visual Studio Code was exercised successfully.

- IntelliSense: offered live completions and parameter hints, reducing typing effort.
- Breakpoints: paused execution exactly at the chosen line so variables could be inspected.
- Refactoring: Rename Symbol updated every instance of a name across the file at once.
- Git Collaboration: the repository was initialised, staged, committed, and linked to a remote.
- Command Palette: editor commands were run quickly from the keyboard.
- Build Modes: Debug supported active troubleshooting while Release targeted optimised performance.

RESULT

The experiment was completed successfully. All of the major Visual Studio Code features, Build configurations, Breakpoints, Refactoring, Git integration, the Command Palette, and IntelliSense, were explored and used effectively, and their contribution to a smooth and collaborative development workflow was understood.

CONCLUSION

The lab provided direct, hands-on experience with the essential capabilities of Visual Studio Code. It is clear that a modern IDE is much more than a text editor; it is an ecosystem that streamlines the whole development process. IntelliSense and the Command Palette accelerate everyday coding, breakpoints and Debug/Release builds simplify debugging and tuning, refactoring protects code quality, and Git integration enables teamwork. Command of these tools is fundamental for anyone working in an Agile Software Development setting.